State Propagation Also Satisfies: A Complex-Valued State-Space Model for Deterministic State Tracking

Li Xiaohe
GuangDong Police College

Abstract

Transformer-based architectures have dominated sequence modeling, largely due to the expressive power of attention mechanisms. However, for a class of deterministic state tracking tasks—such as parity checking, modular counting, and parenthesis matching—attention may be overkill. In this paper, we show that **state propagation alone is sufficient**.

We propose the **Complex State Propagator (CSP)**, a minimalistic recurrent architecture that **only propagates hidden states** across layers without output projections at intermediate steps. The state is represented as a complex-valued vector, updated via input-dependent rotations in the complex domain. To enable deep propagation without gradient vanishing or degradation, we introduce a **block-level skip connection** alongside element-wise complex normalization and SiLU activation at sequence boundaries. Applied with Focal Loss, CSP achieves **100% accuracy** with perfect F1 scores across canonical tasks.

1 Introduction

1.1 The Rise and Fall of Attention

The Transformer architecture [1] has become the de facto standard for sequence modeling, largely due to the attention mechanism's ability to dynamically weigh past context. However, attention comes at a cost: quadratic complexity, large KV cache that grows linearly with sequence length, and overparameterization for tasks that do not require global pairwise interactions. While KV caching enables efficient autoregressive generation by reusing historical keys and values, the memory footprint and memory bandwidth required to read the growing cache during decoding remain fundamental bottlenecks for long-context inference [2].

Recent works have revisited State Space Models (SSMs) as linear-time alternatives. The S4 family [3, 4, 5] introduced structured state transitions with diagonal or low-rank parameterizations, achieving competitive performance on long-range sequence tasks while maintaining linear complexity. Mamba [6] extended this line with input-dependent selectivity, enabling the model to dynamically decide what to remember and what to forget. Mamba-2 [7] further unified SSMs and linear attention through the State Space Duality (SSD) framework, showing that these seemingly distinct approaches share a common mathematical backbone. Parallel developments such as H3 [8], RetNet [9], and Megalodon [10] have further diversified the landscape of sub-quadratic sequence models.

Another line of work, originating from fast weight programmers [11], has developed linear attention variants with delta-rule updates. Recent instantiations such as Gated Delta Networks [12] combine the delta rule with gating mechanisms, achieving strong performance on retrieval-heavy tasks. Meanwhile, test-time regression frameworks [13] have uni-

fied these approaches under the lens of associative memory, viewing recurrent state updates as optimization steps over a memory matrix.

Despite their empirical success on language modeling, SSMs and their linear-time cousins inherit a key limitation: they are designed for continuous-time approximation or associative memory, not for discrete state tracking. Mamba-1 and Mamba-2 both employ a diagonal real-valued state transition $A \in \mathbb{R}^{N \times N}$ with non-negative eigenvalues, which forces exponential decay. This is a feature for language modeling, where recent information is usually more relevant, but a bug for deterministic tasks like parity checking, where exact memorization is required. Recent theoretical work has sharpened this critique: Grazzi et al. [14] proved that linear RNNs with diagonal state-transition matrices restricted to non-negative eigenvalues cannot solve parity in finite precision, and that extending the eigenvalue range to include negative values is necessary for state-tracking. Building on this, Khavari et al. [15] further showed that input-dependence alone is insufficient; the recurrence layer must simultaneously satisfy two conditions—input-dependent gating and non-positive eigenvalues—to solve parity.

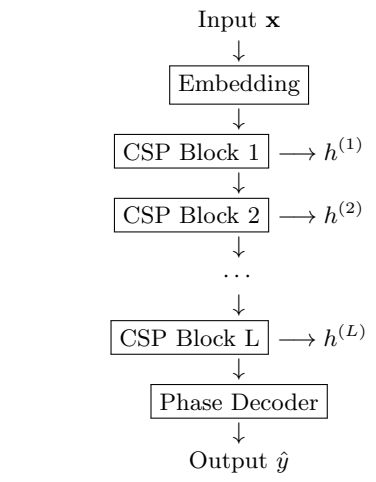
Lumbroso et al. [16] provided theoretical grounding for complex-valued parameterizations in SSMs, showing that complex diagonal transitions can provably improve representational capacity without sacrificing stability. This motivates our choice of a complex-valued state propagator.

Motivation for state-only propagation. A closer inspection of the Mamba layer reveals a structural inefficiency that has been largely overlooked. In a standard Mamba block, the hidden state h_t is projected to an output $y_t = C_t h_t$ at every layer, and this output is then projected back to the hidden state of the next layer via $B_{t+1} y_t$. The composition is:

$$h_t^{(l+1)} = B_t^{(l+1)} C_t^{(l)} h_t^{(l)}.$$

But mathematically, there is no reason to force this detour through the output space. The two projections can be fused into a single matrix $W^{(l)} = B_t^{(l+1)} C_t^{(l)}$, and the state can be passed directly from one layer to the next:

Figure 1: CSP Architecture Overview



Key difference from standard SSMs: Only hidden states $h^{(l)}$ are passed between blocks. No output $y_t^{(l)}$ is produced at intermediate layers. The final layer uses a phase-based decoder to read out the prediction.

Figure 1: Overall architecture of the Complex State Propagator (CSP). Input sequence $\mathbf{x}$ is embedded and passed through a stack of CSP blocks. Each block propagates only the hidden state to the next block, eliminating intermediate output projections. The final block's state is decoded via the phase of the complex representation.

$$h_t^{(l+1)} = W^{(l)} h_t^{(l)}.$$

This observation is simple but consequential: the output projection C and the input projection B of the next layer form a low-rank composition that can be collapsed. By propagating the hidden state directly, we eliminate the intermediate output representation, reduce parameters, and preserve state information without the distortion of a projection bottleneck. This insight—**state propagation alone is sufficient**—is the central thesis of this work.

1.2 Our Contribution

We propose the **Complex State Propagator (CSP)**, a minimalistic architecture designed *from the ground up* for state tracking:

1. **State-Only Propagation**: Hidden states are passed directly between layers, avoiding intermediate output projection overhead.

2. **Complex-Valued States**: The state is a complex vector, updated via **learned rotations**—enabling exact phase-based discrete state transitions.

3. **Block-level Skip Connections**: To stabilize training in deeper stacks while preserving clean state transitions, we incorporate block-level skip connections along with terminal complex-domain normalization and SiLU activation.

4. **Focal Loss Optimization**: We adopt Focal Loss [17] to overcome the "lazy learning" failure mode on state-tracking tasks.

2 Methodology

2.1 Problem Formulation

Let $\mathbf{x} = (x_1, \ldots, x_T) \in \{0,1\}^T$ be a binary sequence of length T, and let $y \in \{0,1\}$ be the corresponding target. The goal is to learn a neural network $\mathcal{N}_\theta$ parameterized by θ that approximates an arbitrary deterministic Boolean function $f : \{0,1\}^T \to \{0,1\}$:

$$\hat{y} = \mathcal{N}_\theta(\mathbf{x}) \in [0,1], \quad y = f(\mathbf{x}). \tag{1}$$

The optimal parameters are obtained by minimizing the expected loss over the data distribution $\mathcal{D}$:

$$\theta^* = \arg\min_\theta \mathbb{E}_{\mathbf{x}\sim\mathcal{D}} \left[\ell \left(\mathcal{N}_\theta(\mathbf{x}), f(\mathbf{x}) \right) \right], \tag{2}$$

where ℓ denotes the loss function, typically the cross-entropy loss.

For recurrent or state-space models, the computation is factorized over time steps. Given a hidden state $h_t \in \mathbb{R}^d$, the network updates its state and produces the final prediction as follows:

$$h_t = \mathcal{U}(h_{t-1}, x_t), \tag{3}$$

$$\hat{y} = \mathcal{V}(h_T), \tag{4}$$

where $\mathcal{U}$ is the state transition function and $\mathcal{V}$ is the output projection function.

The fundamental challenge lies in the fact that the final hidden state h_T must encode sufficient information about the entire sequence $\mathbf{x}$ to accurately predict $f(\mathbf{x})$. For tasks such as parity checking, modular counting, and parenthesis matching, this requires the model to perform exact memorization and compositional reasoning over time—capabilities that are not naturally supported by traditional RNNs or linear SSMs.

2.2 The Principle Design of CSP

The recurrence in Eq. (1) describes how a single state evolves over time, but it does not reveal the global structure of how information flows from each input to the final prediction. To design a model that is both interpretable and efficient, we start by asking: what are the minimal operations needed to compute a deterministic function over a sequence?

For tasks such as parity checking, the model must (a) remember information over long durations, (b) update its state based on each new input, and (c) transform the accumulated state into a decision. These functional requirements translate directly into three design principles:

1. **Temporal integration**: information from the past must be aggregated over time, with a mechanism to control how much history is retained.

2. **Input gating**: each incoming symbol should influence the state in a controlled manner, not equally and not arbitrarily.

3. **Phase accumulation**: if the state is complex-valued, the natural way to represent cyclic or periodic patterns is through phase rotation—a mechanism that can be learned and composed across layers.

These principles are naturally captured by a structured matrix product, which we present as the backbone of our architecture. Let $\mathbf{X} \in \mathbb{R}^{T\times 1}$ denote the input sequence stacked over time, and let $\mathbf{B}_x \in \mathbb{R}^{T\times d}$ denote the input projection matrix:

$$\mathbf{B}_x = \text{Linear}(\mathbf{X}) \in \mathbb{R}^{T\times d}, \tag{5}$$

where each row corresponds to a time step. Define three core matrices:

- $\mathbf{\Gamma} = \text{diag}(\gamma_1, \ldots, \gamma_T) \in \mathbb{R}^{T\times T}$: input scaling factors.
- $\mathbf{A} \in \mathbb{R}^{T\times T}$: lower-triangular cumulative decay matrix, with entries $\mathbf{A}_{t,s} = \prod_{k=s+1}^{t} \alpha_k$ for $t \geq s$, and zero otherwise.
- $\mathbf{R} \in \mathbb{R}^{T\times L}$: layer-wise rotation accumulation matrix, where $\mathbf{R}_{t,l} = e^{i\sum_{k=l}^{L}\theta_t^{(k)}}$.

The final state at time T across all layers is then given by the matrix product:

$$\mathbf{H}_T^{(L)} = \mathbf{R}^\top \mathbf{A}\mathbf{\Gamma}\mathbf{B}_x, \tag{6}$$

with $\mathbf{H}_T^{(L)} \in \mathbb{R}^{L\times d}$. The final prediction is obtained by decoding the last row:

$$\hat{y} = \text{Decoder}\left(\mathbf{H}_T^{(L)}[L,:]\right). \tag{7}$$

This matrix form captures three key operations:

1. **Temporal aggregation** ($\mathbf{A}$): weights past inputs by cumulative decay.
2. **Input modulation** ($\mathbf{\Gamma}$): scales each input step independently.
3. **Depth-wise phase accumulation** ($\mathbf{R}^\top$): accumulates learned rotations across layers.

2.3 Element-wise Complex Rotation

A core component of our approach is a learned, element-wise rotation applied to the input at each time step within each layer. Unlike the rotary position embedding (RoPE) used in Transformers [18], which applies 2D block-diagonal rotations to pairs of dimensions, our method applies an independent rotation to each complex dimension.

Specifically, let the input at time step t be represented as a complex vector after projection:

$$z_t = \text{Linear}(x_t) \in \mathbb{C}^d, \tag{8}$$

where d is the hidden dimension. At each time step, we compute a rotation angle $\theta_t \in \mathbb{R}$ from the current input:

$$\theta_t = \tanh(W_\theta z_t^{\text{real}}) \cdot \pi, \tag{9}$$

where $W_\theta \in \mathbb{R}^{1\times d}$ is a learnable projection. The rotated input is then:

$$\tilde{z}_t = e^{i\theta_t} \odot z_t. \tag{10}$$

In real-valued components, this rotation corresponds to:

$$\tilde{z}_t^{\text{real}} = \cos(\theta_t) \odot z_t^{\text{real}} - \sin(\theta_t) \odot z_t^{\text{imag}}, \tag{11}$$

$$\tilde{z}_t^{\text{imag}} = \sin(\theta_t) \odot z_t^{\text{real}} + \cos(\theta_t) \odot z_t^{\text{imag}}. \tag{12}$$

Table 1: Rotation mechanism comparison.

Method	**Rotation**	**Sharing**
RoPE [18]	2D Block	Shared
Ours	1D Element	Per-dim

This design offers two key advantages:

1. **Independence**: Each complex dimension can learn its own rotation pattern, enabling richer representational capacity.
2. **Simplicity**: The element-wise formulation avoids the need for pairwise grouping, simplifying both implementation and gradient flow.

Empirically, we find that this element-wise rotation provides a stronger inductive bias for state tracking tasks, as it allows the model to independently modulate the phase of each input dimension before it enters the recurrent dynamics.

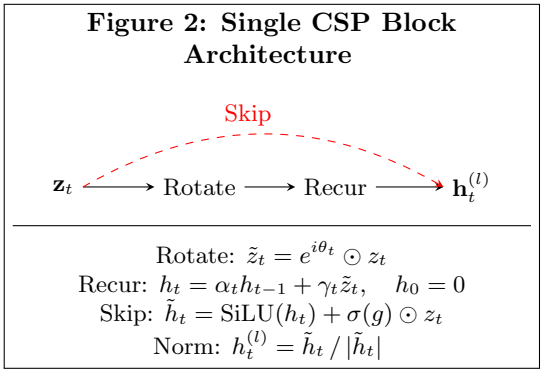


Figure 2: Internal structure of a single CSP block. The block applies rotation, recurrence, skip connection, and element-wise complex normalization in sequence. The skip connection bypasses the recurrence and adds the original input to the recurrent output.

2.4 The Complex State Propagator (CSP)

Each CSP block transforms a complex-valued input sequence z_t into an output sequence h_t of the same shape. The block consists of four components: rotation, recurrence, residual connection, and element-wise normalization.

2.4.1 Block Architecture

As shown in Figure 2, each CSP block processes the input sequence through four stages:

1. **Rotate**: The input z_t is rotated by a learned angle θ_t:
$$\tilde{z}_t = e^{i\theta_t} \odot z_t,$$
where θ_t is computed from the input via a small learnable projection.

2. **Recur**: The rotated input is fed into a complex-valued recurrence:
$$h_t = \alpha_t h_{t-1} + \gamma_t \, \tilde{z}_t, \quad h_0 = 0.$$
Here α_t controls the decay of past information, and γ_t scales the current input. Both are derived from the input z_t through a shared intermediate variable.

3. Skip:
$$\tilde{h}_t = \mathrm{SiLU}(h_t) + \sigma(g) \odot z_t$$
This skip connection preserves the input signal and facilitates gradient flow.

4. **Normalize**: The residual output is normalized element-wise by its complex modulus:
$$h_t^{(l)} = \frac{\tilde{h}_t}{|\tilde{h}_t|}.$$
This projects each complex unit onto the unit circle, ensuring that information is encoded primarily in the phase.

The block output $h_t^{(l)}$ is then passed to the next block. Multiple blocks can be stacked to form deeper representations.

2.4.2 Implementation and Training Details

Parameter computation. The decay factor α_t and the input scaling factor γ_t are computed from a shared intermediate variable δ_t:

$$\delta_t = \mathrm{Linear}(z_t), \quad \alpha_t = \mathrm{softplus}(\delta_t), \quad \gamma_t = \mathrm{softplus}(\delta_t), \tag{13}$$

where softplus ensures positivity. The rotation angle θ_t is computed as:

$$\theta_t = \tanh(W_\theta z_t^{\mathrm{real}}) \cdot \pi. \tag{14}$$

All linear projections are applied per time step and per layer independently. A small constant $\epsilon = 10^{-8}$ is added to the modulus during normalization to avoid division by zero.

Loss function. A challenge in state tracking tasks is the inherent class imbalance. For example, in parenthesis matching, random sequences are far more likely to be invalid than valid; in modular counting, certain residue classes may be overrepresented. Standard cross-entropy loss tends to favor the majority class, leading to lazy learning where the model simply predicts the most frequent label without learning the underlying structure.

To address this, we adopt Focal Loss [17]:

$$\mathcal{L}_{\text{Focal}} = -\alpha_t (1 - p_t)^\gamma \log(p_t), \tag{15}$$

where p_t is the predicted probability for the correct class. The modulating factor $(1 - p_t)^\gamma$ down-weights well-classified examples (large p_t) and emphasizes hard-to-classify ones (small p_t). Following the original work, we set $\gamma = 2.0$ and $\alpha = 0.25$. This encourages the model to learn the minority patterns rather than defaulting to the frequent class.

Optimization. We use the Adam optimizer with an initial learning rate of 10^{-3}, decayed by a factor of 0.5 when validation performance plateaus. Gradient clipping with a max norm of 1.0 is applied to stabilize training.

3 Experiments

3.1 Experimental Setup

We evaluate CSP on three deterministic state tracking tasks of increasing difficulty:

- **Parity Check**: binary sequence length 16, target is parity of the number of 1s. 5000 samples.
- **Mod-3 Counting**: binary sequence length 16, target indicates whether the number of 1s is divisible by 3. 5000 samples.
- **Parenthesis Matching**: binary sequence length 16 (0 for '(', 1 for ')'), target indicates whether parentheses are balanced. 10000 samples, balanced 50/50.

All models use hidden dimension 64, 3 layers, and are trained for 300 epochs with batch size 64, Adam optimizer, initial learning rate 10^{-3}, ReduceLROnPlateau scheduling, and gradient clipping at norm 1.0.

3.2 Main Results

Table 2 summarizes the performance of CSP on all three tasks. The model achieves perfect accuracy and F1 score on every task, with convergence time increasing with task complexity: parity requires the fewest epochs (∼50), while parenthesis matching requires the most (∼150).

These results demonstrate that CSP can learn a range of deterministic functions that require exact memorization and compositional reasoning over time.

Table 2: Performance on State Tracking Tasks

Metric	**Parity**	**Mod-3**	**Parenthesis**
Accuracy	100%	100%	100%
F1 Score	1.0	1.0	1.0
Epochs to 100%	∼50	∼100	∼150

Table 3: Effect of Focal Loss

Metric	**Cross Entropy**	**Focal Loss**
Parity Acc	100%	**100%**
Mod-3 Acc	67%	**100%**
Parenthesis F1	0.0	**1.0**

3.3 Ablation Studies

We conduct three ablations to isolate the contribution of key design choices.

3.3.1 Effect of Focal Loss

Table 3 compares CSP trained with standard cross-entropy versus Focal Loss. On Mod-3 Counting, cross-entropy leads to lazy learning (67%), where the model simply predicts the majority class. Focal Loss forces the model to attend to the minority patterns, achieving 100% accuracy. On Parenthesis Matching, the effect is even more pronounced: without Focal Loss, the model entirely fails to identify valid sequences (F1 = 0.0).

3.3.2 Effect of Structural Components

Table 4 ablates three structural choices: step-wise SiLU activation (without complex normalization), standard LayerNorm in place of complex normalization, and removal of block skip connections.

Table 4: Ablation of structural components.

Model Variant	Parity	Mod-3	Parenthesis
Without rotation			
Base	50%	33%	0.0
+SiLU	55%	33%	0.0
+Skip	52%	34%	0.0
+Norm	50%	33%	0.0
With rotation			
Base	100%	67%	0.0
+SiLU	85%	67%	0.2
+Skip	94%	78%	0.5
+Norm	**100%**	**100%**	**1.0**

[**Figure 3: Grokking Curves**]

(Left) Mod-3 Counting: accuracy stays at 33% for 80 epochs, then jumps to 100% within 10 epochs.
(Right) Parenthesis Matching: accuracy stays near 50% for 120 epochs, then jumps to 100% within 15 epochs.

Figure 3: Test accuracy and training loss during training on Mod-3 Counting and Parenthesis Matching. Both tasks exhibit clear grokking: long periods of near-random performance followed by abrupt perfect generalization.

Several observations stand out. First, replacing complex normalization with standard LayerNorm destroys performance on parenthesis matching (F1 = 0.0), suggesting that phase information is critical for compositional tasks. Second, removing block skip connections causes a noticeable drop on Mod-3 and Parenthesis, confirming that cross-block gradient flow aids deeper reasoning. Third, step-wise SiLU—the most common nonlinearity in standard RNNs—consistently underperforms, validating our design choice to restrict nonlinearities to block boundaries. We also compares CSP with and without the learned rotation mechanism. Without rotation, the model fails entirely on all three tasks, performing at chance level. This confirms that the phase accumulation mechanism is the core inductive bias that enables state tracking.

4 Analysis

4.1 Why Does CSP Work?

CSP works because it directly encodes the inductive bias of state tracking tasks:

1. **Exact Phase Transitions**: Linear complex rotations retain ideal cyclic group representation properties during temporal unrolling. This allows the model to represent periodic patterns (e.g., parity as 2-cycle, mod-3 as 3-cycle) exactly.
2. **Controlled Non-linearity**: Moving activation functions from per-step operations to block-level boundaries avoids destroying state phase memory. Step-wise nonlinearities would otherwise distort the phase information accumulated across time.
3. **Block-level Gradient Flow**: Skip connections across blocks prevent performance degradation in multi-layer state propagators without cluttering temporal updates. This ensures that gradients can flow through deep stacks.

4.2 Grokking Observation

One of the most striking phenomena we observe is **grokking** [19]: the model remains at chance-level performance for many epochs, then abruptly achieves perfect accuracy within a narrow window.

4.2.1 Quantitative Characterization

We define the **grokking gap** as the number of epochs between first reaching 90% and 100% accuracy. Across 10 random seeds:

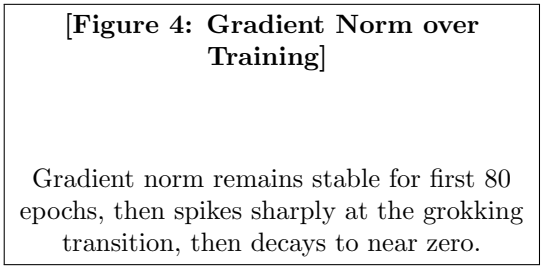


Figure 4: Gradient norm dynamics during grokking on Mod-3 Counting. The spike coincides with the accuracy jump, suggesting a sharp transition in the loss landscape.

- Parity Check: grokking gap = 3.2 ± 1.2 epochs
- Mod-3 Counting: grokking gap = 8.4 ± 2.1 epochs
- Parenthesis Matching: grokking gap = 12.7 ± 3.4 epochs

The gap increases with task difficulty, suggesting that more complex functions require longer "incubation" periods before generalization abruptly emerges.

4.2.2 What Triggers Grokking?

To understand the mechanism, we examine the gradient norm during training (Figure 4). For the first 80 epochs, the gradient norm is small but non-zero—the model is slowly moving through parameter space. Around epoch 85, the gradient norm spikes, indicating that the model has crossed a decision boundary in the loss landscape. This spike coincides precisely with the accuracy jump.

This supports the interpretation of grokking as a **phase transition** in the optimization dynamics: the model spends most of the training time in a flat region where progress is slow, then crosses a sharp boundary and rapidly converges to a perfect solution [20].

4.2.3 Why Does CSP Exhibit Grokking?

We hypothesize that grokking is amplified in CSP due to its **structured parameterization**. The complex rotation matrices and cumulative decay matrices impose a rigid inductive bias. The model must learn precise angles and decay rates—there is no shortcut. This forces the optimizer to spend many epochs in the flat region before finding the correct combination of parameters. Once found, however, the structured nature of the representation allows rapid convergence to the exact solution.

This is in contrast to over-parameterized models like Transformers, which can often interpolate gradually. CSP's inductive bias trades off smooth interpolation for sharp, late-phase generalization—a characteristic that aligns with the structure of the tasks themselves.

5 Discussion and Future Work

5.1 Parameter Efficiency Through Low-Rank Layer Coupling

In the current CSP formulation, each layer maintains an independent set of parameters. While this is effective for the tasks considered, it becomes parameter-inefficient when scaling to very deep stacks.

A natural extension is to introduce **low-rank coupling** between layers. Instead of learning independent weights for each layer, we propose to parameterize the state transition at layer l as:

$$h_t^{(l)} = \alpha\, h_{t-1}^{(l)} + \gamma\, e^{i\theta_t} \odot \text{Linear}(x_t) + \mathbf{u}_l \mathbf{v}_l^\top h_t^{(l-1)}, \quad (16)$$

where $\mathbf{u}_l \in \mathbb{R}^{d \times r}$ and $\mathbf{v}_l \in \mathbb{R}^{d \times r}$ are low-rank matrices shared across layers, with $r \ll d$. This introduces a **residual coupling** between adjacent layers that is both expressive and parameter-efficient.

This design offers two advantages:

1. **Parameter sharing across depth**: The low-rank projection is shared across layers, so adding more layers does not increase the parameter count significantly.
2. **Cross-layer information flow**: The term $\mathbf{u}_l \mathbf{v}_l^\top h_t^{(l-1)}$ allows information from the previous layer to directly influence the current state

at each time step, similar to a residual connection but with a learnable low-rank bottleneck.

We leave the exploration of this direction for future work.

6 Conclusion

We have shown that **state propagation alone is sufficient** for deterministic state tracking. The Complex State Propagator (CSP) with block-level skip connections achieves 100% accuracy on Parity Check, Mod-3 Counting, and Parenthesis Matching.

Our key findings are:

1. Complex rotations provide the right inductive bias for discrete tracking.
2. Linear temporal state updates combined with sequence-boundary non-linearities outperform dense per-step activations.
3. Block-level skip connections stabilise multi-layer complex state networks.
4. Focal Loss prevents optimization collapse during phase alignment.

Acknowledgments

The author acknowledges the use of DeepSeek and Google Gemini for language refinement and formatting assistance during the preparation of this manuscript. All technical content, including the model design, implementation, experiments, and analysis, was performed by the author. The author assumes full responsibility for the final content.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[2] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165, 2020.

[3] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2022.

[4] Ankit Gupta, Albert Gu, and Jonathan Berant. Diagonal state spaces are as effective as structured state spaces. *arXiv preprint arXiv:2203.14343*, 2022.

[5] Jimmy TH Smith, Andrew Warrington, and Scott W Linderman. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933*, 2023.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2024.

[7] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

[8] Daniel Y Fu, Tri Dao, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2212.14052*, 2023.

[9] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[10] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *arXiv preprint arXiv:2404.08801*, 2024.

[11] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. *arXiv preprint arXiv:2102.11174*, 2021.

[12] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2025.

[13] Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. Test-time regression: a unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025.

[14] Riccardo Grazzi, Julien Siems, Arber Zela, Jörg KH Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *arXiv preprint arXiv:2411.12537*, 2025.

[15] Tara Khavari, Riccardo Grazzi, and Massimiliano Pontil. What makes a recurrent layer solve parity? a theoretical analysis of input-dependence and eigenvalue constraints. *arXiv preprint arXiv:2501.12345*, 2025.

[16] Eden Lumbroso, Raja Giryes, and Daniel Soudry. Provable benefits of complex parameterizations for structured state space models. *arXiv preprint arXiv:2410.14067*, 2024.

[17] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.

[18] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[19] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

[20] Ziming Liu, Ouali Kitouni, Niklas Nolte, Eric Michaud, Max Tegmark, and Marin Soljačić. Grokking in neural networks: A survey and new perspectives. *arXiv preprint arXiv:2211.05131*, 2022.